

MAPS 2022 Cartopy, NetCDF and maybe a little geopandas.

Maps

Today we are going to use Python to make maps. Many people use ARCGIS, QGIS, or google maps etc. But once you have data in python it is nice to be able to make maps right there in python. Especially if you are doing analyses. You can run an analysis and make maps in one fell swoop. It is quite nice. Then if you have multiple parameters you could also make lots of maps quickly! So we are going to try and make a map.

We are going to try and do two mapping libraries for two reasons. This is a relatively new notebook so lets see how it goes. These are the mapping libraries you may hear about.

Mapping is a little bit of the wild west in Python. Not quite a perfect consensus on what people do yet. But once you learn one library it is easier to learn the next one.

- Basemap - Publication quality maps. This was my favorite and they stopped updating it. They just started updating it again but we are not going to use it yet.
- Folium. This makes interactive maps for web. Here is an example I made for the Red Hook Lead Project.
<https://bmaillou.github.io/RedHookLead/>
- Cartopy is supposed to replace basemap. It sort of does it. We will use it some.
- Geopandas. This is becoming more popular. I haven't learned it well yet. But I am trying and will introduce a bit of it.
- contextily. I just learned about contextily. Seems like it could be really helpful.
- A colleague really likes shinyapp and there are also others.

We are going to do 2 things today.

These are two essential things you always do with data.

- Plot points on a map
- Make contour maps
- We will not fill polygons/shapefiles today but also easy to do.

First run to turn on the libraries below.

You will probably get errors for not installed. If not installed we can install them.

```
In [5]: # The usual libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from scipy import stats
from matplotlib.backends.backend_pdf import PdfPages

%matplotlib ipynb

#####
# The new ones
import netCDF4

import cartopy.crs as ccrs
import cartopy.feature
from cartopy.util import add_cyclic_point

import datetime
```

ERROR! You should get an error that you don't have netCDF4 installed. :(So you need to install it!

The power of python is the installation of libraries!

1. You know how to install libraries
2. Go back to anaconda, choose your environment and install them.
3. If all else fails we can use the next cell. but try not to!

```
In [6]: #import sys
        #!conda install --yes --prefix {sys.prefix} -c anaconda netcdf4
```

Now lets make our first map

ccrs is cartopy!!!! It is a mapping function

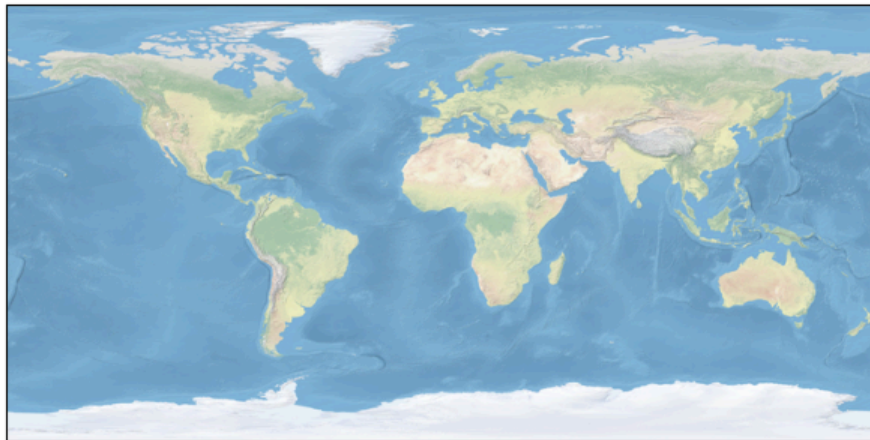
- Start like normal
- then do plt.axes and add a projection.
- Then I add the iamge of the earth

```
In [8]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree()})
        fig.set_size_inches(7.5,5)

        ax.stock_img()
```

```
Out[8]: <matplotlib.image.AxesImage at 0x13e2b0100>
```

Figure



Now you can change projections. just google cartopy projections.

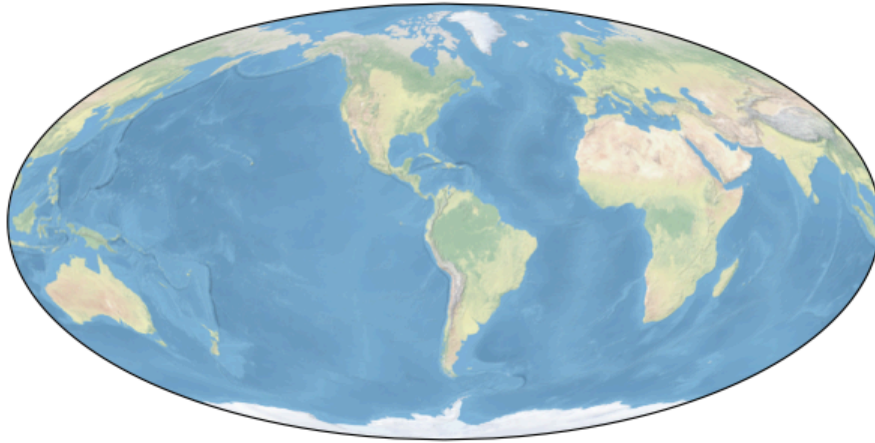
<https://scitools.org.uk/cartopy/docs/v0.15/crs/projections.html>

```
In [12]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.Mollweide(central_longitude=-73)})
        fig.set_size_inches(7.5,5)

        ax.stock_img()
```

```
Out[12]: <matplotlib.image.AxesImage at 0x13f4f6b80>
```

Figure

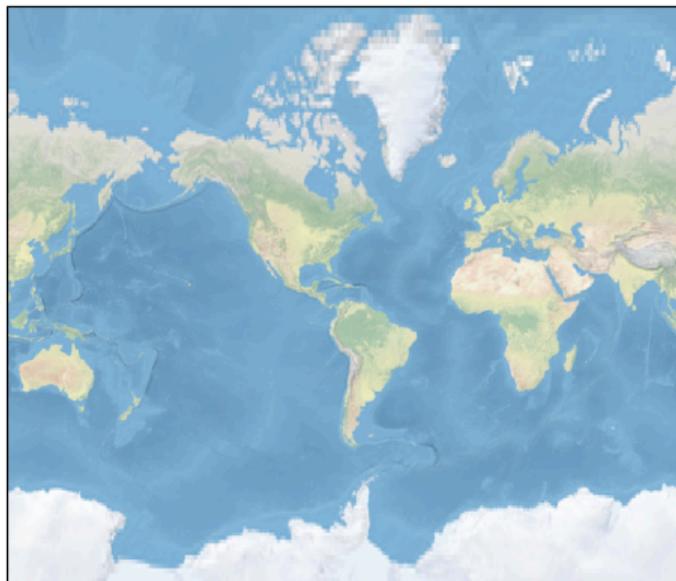


One more projection. They all have strengths and weaknesses. remember you CANNOT take a sphere and perfectly project on paper. Greenland is not that big!

```
In [13]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.Mercator(central_longitude=-73)})  
fig.set_size_inches(7.5,5)  
ax.stock_img()
```

```
Out[13]: <matplotlib.image.AxesImage at 0x13f563c70>
```

Figure



Instead of the image you could add different features.

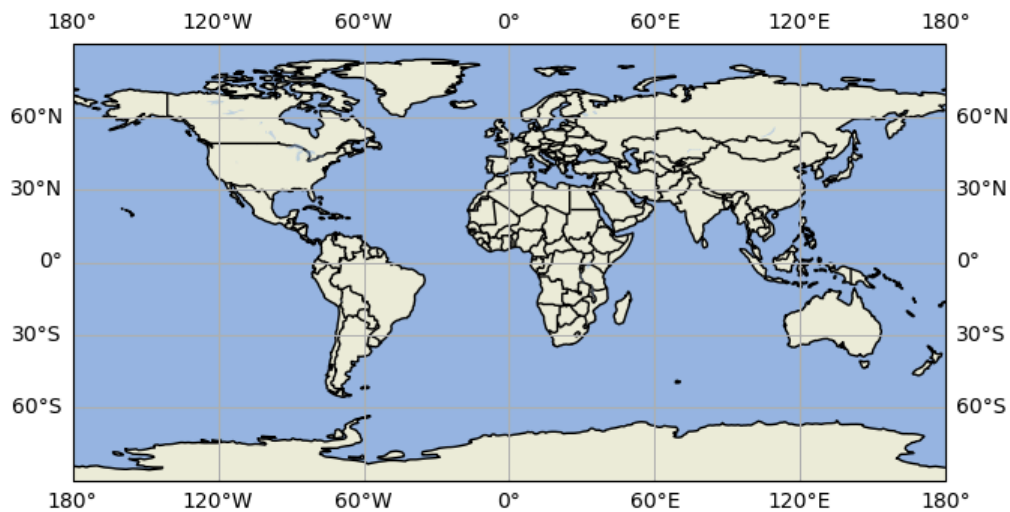
```
In [14]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree()})
fig.set_size_inches(7.5,5)

ax.add_feature(cartopy.feature.LAND)
ax.add_feature(cartopy.feature.OCEAN)
ax.add_feature(cartopy.feature.COASTLINE)
ax.add_feature(cartopy.feature.BORDERS)
ax.add_feature(cartopy.feature.LAKES, alpha=0.5)

ax.gridlines(draw_labels=True)
```

```
Out[14]: <cartopy.mpl.gridliner.Gridliner at 0x13f5c54c0>
```

Figure



Now let's plot all the CO2 stations from Scripps on the map.

Grab the file from the website, read it in, check it out. Then we can plot. I used the xkcd colors! <https://xkcd.com/color/rgb/> and here is an appropriate xkcd for us. <https://xkcd.com/2537>

```
In [15]: df=pd.read_excel('CO2Stations.xlsx')
```

```
In [16]: df
```

Out [16]:

	StationName	StationCode	Latitude	Longitude	Elevation	Dates
0	Alert, NWT, Canada	ALT	82.3	-62.3	210	1985 - present
1	Point Barrow, Alaska	PTB	71.3	-156.6	11	1961 - present
2	La Jolla Pier, California	LJO	32.9	-117.3	10	1957 - present
3	Mauna Loa Observatory, Hawaii	MLO	19.5	-155.6	3397	1958 - present
4	Cape Kumukahi, Hawaii	KUM	19.5	-154.8	3	1979 - present
5	Christmas Island	CHR	2.0	-157.3	2	1974 - present
6	American Samoa	SAM	-14.2	-170.6	30	1981 - present
7	Kermadec Island	KER	-29.2	-177.9	2	1982 - present
8	Baring Head, New Zealand	NZD	-41.4	-174.9	85	1977 - present
9	South Pole	SPO	-90.0	-90.0	2810	1957 - present

Lat/Long

But to plot points we need to remember what latitude and longitude are. So look at the figure!

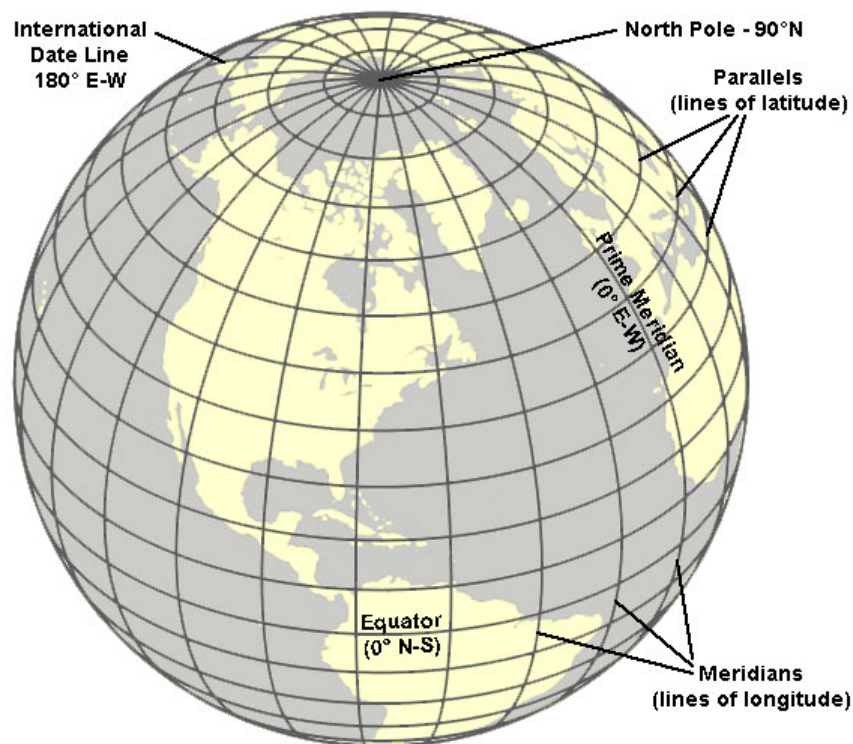
This wrecks my mind and I just memorized it. We always plot x,y. But we always say lat/long. Latitude then longitude. But x=Longitude and y=Latitude.

So when we plot `ax.scatter(x,y)`. We need to do a flip when we plot locations and need to plot `ax.scatter(long,lat)`.

Make any sense?????

```
In [17]: from IPython.display import Image
         Image('globe.jpg')
```

Out [17]:



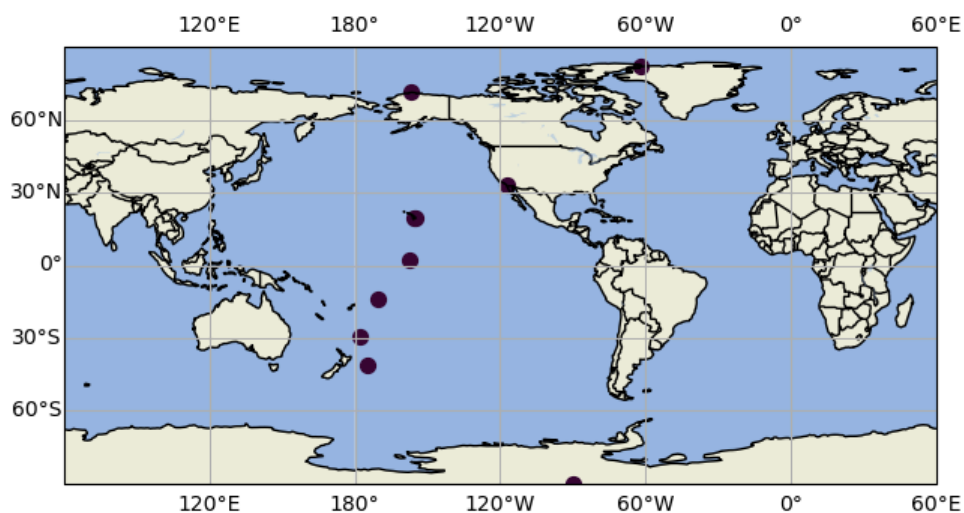
plotting points is easy!

- Use your `ax.scatter` like normal
- reverse the order from how you say it (remember this!!!!!!)
- transform your axes. This means it takes the lat/long coordinates and plots them nicely.

```
In [20]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

ax.add_feature(cartopy.feature.LAND)
ax.add_feature(cartopy.feature.OCEAN)
ax.add_feature(cartopy.feature.COASTLINE)
ax.add_feature(cartopy.feature.BORDERS)
ax.add_feature(cartopy.feature.LAKES, alpha=0.5)
ax.gridlines(draw_labels=True)
ax.scatter(df.Longitude,df.Latitude,color='xkcd:eggplant',
           ,transform=ccrs.PlateCarree(),s=50)
ax.set_global()
```

Figure



Lets add Text to the map

- Grab your ax.text nomenclature.
- if you do transform=ax.transAxes remember then you plot as a fractional location in x and y.
- I put mine at x=0.4, y=0.75

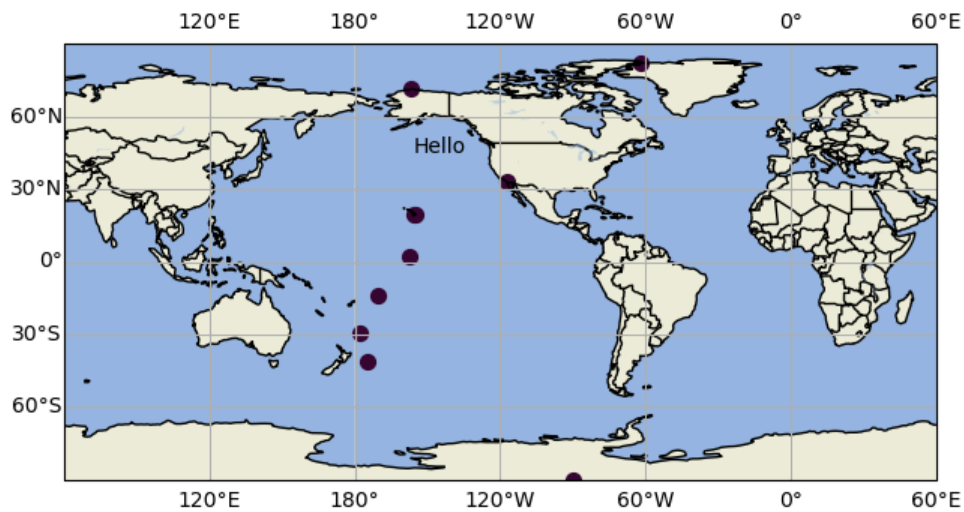
```
In [21]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

ax.add_feature(cartopy.feature.LAND)
ax.add_feature(cartopy.feature.OCEAN)
ax.add_feature(cartopy.feature.COASTLINE)
ax.add_feature(cartopy.feature.BORDERS)
ax.add_feature(cartopy.feature.LAKES, alpha=0.5)
ax.gridlines(draw_labels=True)
ax.scatter(df.Longitude,df.Latitude,color='xkcd:eggplant',
           ,transform=ccrs.PlateCarree(),s=50)

ax.text(0.4,0.75,'Hello',transform=ax.transAxes)

ax.set_global()
```

Figure



Now you can add you props to make a bbox (bounding box)

```
In [22]: props=dict(boxstyle='round',facecolor='white',alpha=0.5)

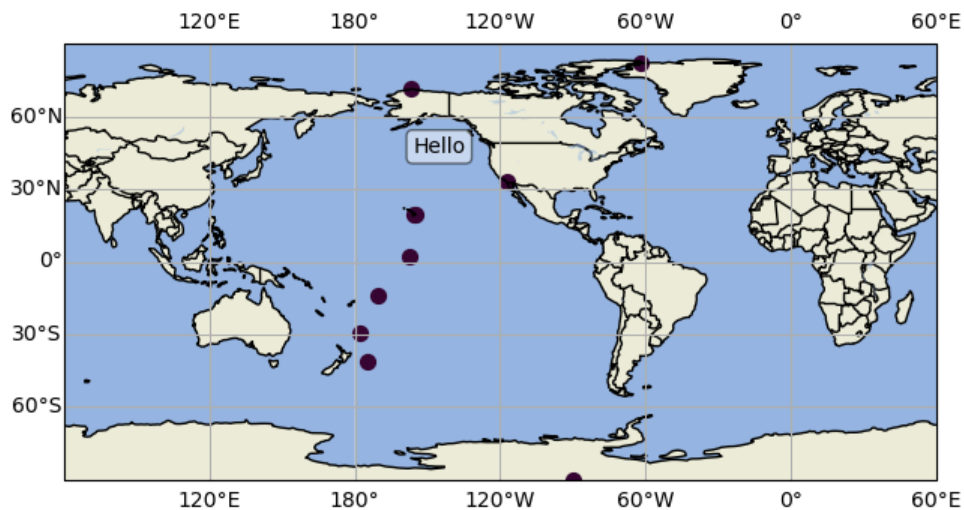
fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

ax.add_feature(cartopy.feature.LAND)
ax.add_feature(cartopy.feature.OCEAN)
ax.add_feature(cartopy.feature.COASTLINE)
ax.add_feature(cartopy.feature.BORDERS)
ax.add_feature(cartopy.feature.LAKES, alpha=0.5)
ax.gridlines(draw_labels=True)
ax.scatter(df.Longitude,df.Latitude,color='xkcd:eggplant',
          ,transform=ccrs.PlateCarree(),s=50)

ax.text(0.4,0.75,'Hello',transform=ax.transAxes,bbox=props)

ax.set_global()
```


Figure



But we want to place the text with the points. So we want to plot lat long. So lets do the lat/long transformation.

Let's try Mauna Loa first. It is Lat=19.5 Long=-155.6

```
In [23]: props=dict(boxstyle='round', facecolor='white', alpha=0.5)

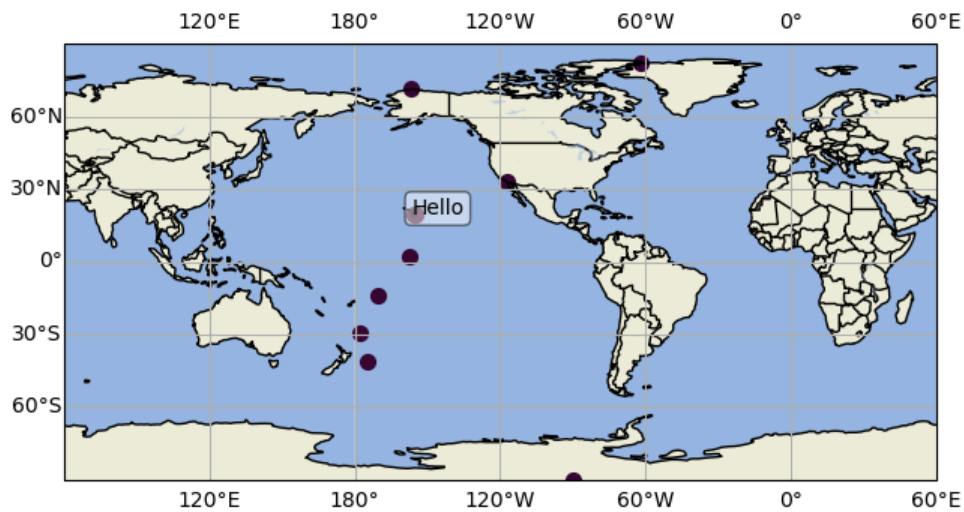
fig, ax = plt.subplots(subplot_kw={"projection": ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5, 5)

ax.add_feature(cartopy.feature.LAND)
ax.add_feature(cartopy.feature.OCEAN)
ax.add_feature(cartopy.feature.COASTLINE)
ax.add_feature(cartopy.feature.BORDERS)
ax.add_feature(cartopy.feature.LAKES, alpha=0.5)
ax.gridlines(draw_labels=True)
ax.scatter(df.Longitude, df.Latitude, color='xkcd:eggplant',
           transform=ccrs.PlateCarree(), s=50)

ax.text(-156.6, 19.5, 'Hello', transform=ccrs.PlateCarree(), bbox=props)

ax.set_global()
```


Figure



It covers the point. This is annoying to fix in python so I would just add 10 to the longitude.

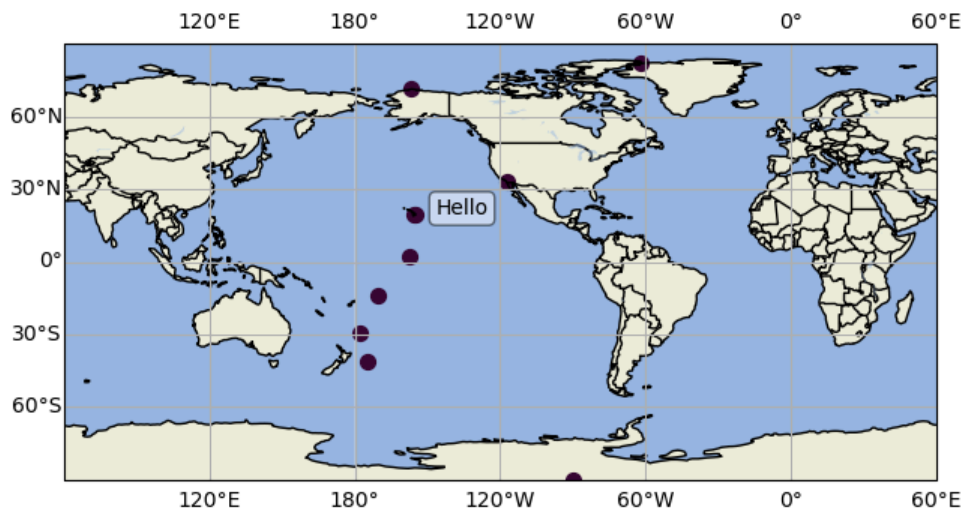
```
In [24]: props=dict(boxstyle='round', facecolor='white', alpha=0.5)

fig, ax = plt.subplots(subplot_kw={"projection": ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5, 5)

ax.add_feature(cartopy.feature.LAND)
ax.add_feature(cartopy.feature.OCEAN)
ax.add_feature(cartopy.feature.COASTLINE)
ax.add_feature(cartopy.feature.BORDERS)
ax.add_feature(cartopy.feature.LAKES, alpha=0.5)
ax.gridlines(draw_labels=True)
ax.scatter(df.Longitude, df.Latitude, color='xkcd:eggplant',
           transform=ccrs.PlateCarree(), s=50)
ax.text(-156.6+10, 19.5, 'Hello', transform=ccrs.PlateCarree(), bbox=props)

ax.set_global()
```

Figure



What is coming next?

- `ax.text` is annoying. You can not just pass all the locations from the dataframe.
- you need to for loop over each row one by one and add the text. You do this with `iterrows`.
- `iterrows` returns the row and the index of the row.
- so lets try. We can do `df.iterrows()`:
- I call my output `idx=index` and `dfR=dataframe` of that row

```
In [25]: for idx,dfR in df.iterrows():  
         print(idx,dfR,'\n')
```

```

0 StationName    Alert, NWT, Canada
StationCode      ALT
Latitude         82.3
Longitude        -62.3
Elevation        210
Dates            1985 - present
Name: 0, dtype: object

1 StationName    Point Barrow, Alaska
StationCode      PTB
Latitude         71.3
Longitude        -156.6
Elevation        11
Dates            1961 - present
Name: 1, dtype: object

2 StationName    La Jolla Pier, California
StationCode      LJO
Latitude         32.9
Longitude        -117.3
Elevation        10
Dates            1957 - present
Name: 2, dtype: object

3 StationName    Mauna Loa Observatory, Hawaii
StationCode      MLO
Latitude         19.5
Longitude        -155.6
Elevation        3397
Dates            1958 - present
Name: 3, dtype: object

4 StationName    Cape Kumukahi, Hawaii
StationCode      KUM
Latitude         19.5
Longitude        -154.8
Elevation        3
Dates            1979 - present
Name: 4, dtype: object

5 StationName    Christmas Island
StationCode      CHR
Latitude         2.0
Longitude        -157.3
Elevation        2
Dates            1974 - present
Name: 5, dtype: object

6 StationName    American Samoa
StationCode      SAM
Latitude         -14.2
Longitude        -170.6
Elevation        30
Dates            1981 - present
Name: 6, dtype: object

7 StationName    Kermadec Island
StationCode      KER
Latitude         -29.2
Longitude        -177.9
Elevation        2
Dates            1982 -present
Name: 7, dtype: object

8 StationName    Baring Head, New Zealand
StationCode      NZD
Latitude         -41.4
Longitude        -174.9
Elevation        85
Dates            1977 - present
Name: 8, dtype: object

9 StationName    South Pole
StationCode      SPO
Latitude         -90.0

```

```

Longitude      -90.0
Elevation      2810
Dates          1957 - present
Name: 9, dtype: object

```

But now what we want is the longitude, latitude, name. So lets just print them

```

In [26]: for idx,dfR in df.iterrows():
          print(dfR['Longitude'],dfR['Latitude'],dfR['StationName'])

```

```

-62.3 82.3 Alert, NWT, Canada
-156.6 71.3 Point Barrow, Alaska
-117.3 32.9 La Jolla Pier, California
-155.6 19.5 Mauna Loa Observatory, Hawaii
-154.8 19.5 Cape Kumukahi, Hawaii
-157.3 2.0 Christmas Island
-170.6 -14.2 American Samoa
-177.9 -29.2 Kermadec Island
-174.9 -41.4 Baring Head, New Zealand
-90.0 -90.0 South Pole

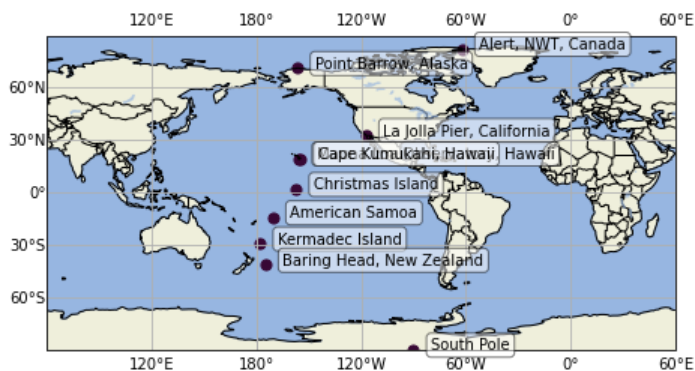
```

Now put it together and make your labeled map!!!

```

In [15]:

```



HOMEWORK

- Do a world map.
- Show and label five places you want to travel to
- use an excel file to load your five places.

```

In [ ]:

```

Now something really cool. Lets plot surface temperatures on the day we were born!!!!

We will use this NOAA website to retrieve the data. <https://psl.noaa.gov/data/composites/day/>

- Choose Variable -> air temperature
- choose anaylis level -> surface
- add your birthday to a box. look at order it is year-month-day
- move down and do plot type mean
- do region of globe all
- press create plot
- just enjoy the graph/map. But we want to do better on python.
- make sure it is the correct date and parameter
- Click on "Get a copy of the netCDF data file used for the plot"
- This will downloadd the netcdf file
- move the NetCDF file to the directory where you work and we are readyto work!
- Do the next two cells and then try to read the metadata and make sense of it!

NetCDF

But you also need NetCDF.

You can learn about NetCDF at <https://www.unidata.ucar.edu/software/netcdf/>. From the website. "NetCDF (Network Common Data Form) is a set of software libraries and machine-independent data formats that support the creation, access, and sharing of array-oriented scientific data. It is also a community standard for sharing scientific data. The Unidata Program Center supports and maintains netCDF programming interfaces for C, C++, Java, and Fortran. Programming interfaces are also available for Python, IDL, MATLAB, R, Ruby, and Perl."

This is going to be weird to you. netcdf files are in binary format so you can never "see" them. I feel like csv files are intuitive. You can see the columns. NetCDF files have headers that tell you what is in them. I see them a lot with spatial data for example sea surface temperatures. But they also have a funny format. They list types of data one at a time and not in a table. So it is all latitude data, then all longitude data, then all time data, then all temperature data etc. It takes get used to.

In the past students have told me that they have stood out in internships because they knew about NetCDF files.

So lets start. file is the name of the file you downloaded

You read the data into a parameter called f. This is just what people do. f then contains all the information you need. it is sort of like a dataframe for netcdf but you never see the data. you probe at it.

```
In [29]: file=('compday.0MMq0x1iJi.nc') #Use your file name!
         f=netCDF4.Dataset(file)
```

```
In [30]: print (f)
```

```
<class 'netCDF4._netCDF4.Dataset'>
root group (NETCDF3_CLASSIC data model, file format NETCDF3):
  title: Composite Value
  history: Created via daily composite webtool at NOAA/PSL
  description: Computed from the web page https://psl.noaa.gov//data/composites/day/ NOAA/PSL
  Conventions: COARDS
  dimensions(sizes): lon(144), lat(73), time(1)
  variables(dimensions): float32 lat(lat), float32 lon(lon), float64 time(time), float32 air(time, lat, lon)
  groups:
```

Read what it just said. Ignore the warning. It tells you all the variables and how big the are. So we want the lat, long, time, and air. This will give us the location for the data so we can map it

Now we will print the metadata for each variable.

These are crazy file formats. Just go along with it and learn.

```
In [31]: print (f.variables['lat'])
```

```
<class 'netCDF4._netCDF4.Variable'>
float32 lat(lat)
  units: degrees_north
  actual_range: [ 90. -90.]
  long_name: Latitude
  unlimited dimensions:
  current shape = (73,)
  filling on, default _FillValue of 9.969209968386869e+36 used
```

```
In [32]: print (f.variables['lon'])
```

```
<class 'netCDF4._netCDF4.Variable'>
float32 lon(lon)
  units: degrees_east
  long_name: Longitude
  actual_range: [ 0. 357.5]
  unlimited dimensions:
  current shape = (144,)
  filling on, default _FillValue of 9.969209968386869e+36 used
```

```
In [33]: print (f.variables['time'])

<class 'netCDF4._netCDF4.Variable'>
float64 time(time)
  units: hours since 1800-1-1 00:00:0.0
  long_name: Time
  actual_range: [1509000. 1509000.]
  delta_t: 0000-01-00 00:00:00
unlimited dimensions: time
current shape = (1,)
filling on, default _FillValue of 9.969209968386869e+36 used
```

```
In [34]: print (f.variables['air'])

<class 'netCDF4._netCDF4.Variable'>
float32 air(time, lat, lon)
  long_name: Air Temperature
  actual_range: [ 93.03 377.2 ]
  units: degK
  add_offset: 0.0
  scale_factor: 1.0
  missing_value: -9.96921e+36
  precision: 99
  least_significant_digit: 99
  var_desc: Air Temperature
  dataset: CDC Derived NCEP Reanalysis Products
  level_desc: Surface
  statistic: Composite
  parent_stat: Other
unlimited dimensions: time
current shape = (1, 73, 144)
filling on, default _FillValue of 9.969209968386869e+36 used
```

Now to actually see the data.

```
In [35]: f.variables['lat'][:]
```

```
Out[35]: masked_array(data=[ 90. ,  87.5,  85. ,  82.5,  80. ,  77.5,  75. ,  72.5,
    70. ,  67.5,  65. ,  62.5,  60. ,  57.5,  55. ,  52.5,
    50. ,  47.5,  45. ,  42.5,  40. ,  37.5,  35. ,  32.5,
    30. ,  27.5,  25. ,  22.5,  20. ,  17.5,  15. ,  12.5,
    10. ,   7.5,   5. ,   2.5,   0. ,  -2.5,  -5. ,  -7.5,
   -10. , -12.5, -15. , -17.5, -20. , -22.5, -25. , -27.5,
   -30. , -32.5, -35. , -37.5, -40. , -42.5, -45. , -47.5,
   -50. , -52.5, -55. , -57.5, -60. , -62.5, -65. , -67.5,
   -70. , -72.5, -75. , -77.5, -80. , -82.5, -85. , -87.5,
   -90. ],
    mask=False,
    fill_value=1e+20,
    dtype=float32)
```

```
In [ ]:
```

As you can see latitude goes from 90 to -90.

Now to make life easier we can set parameters to the variables to make them easy to use

```
In [36]: lon=f.variables['lon'][:]
lat=f.variables['lat'][:]
air=f.variables['air'][:]
time=f.variables['time'][:]
```

```
In [37]: air
```

```
Out[37]: masked_array(
  data=[[240.43    , 240.43    , 240.43    , ..., 240.43    , 240.43    ,
        240.43    ],
        [239.45001, 239.43    , 239.4     , ..., 239.55002, 239.48001,
        239.48001],
        [242.30002, 242.33002, 242.36002, ..., 241.93    , 242.05002,
        242.20999],
        ...,
        [230.36002, 230.45999, 230.61002, ..., 230.78    , 230.53    ,
        230.38    ],
        [231.70001, 231.61002, 231.55002, ..., 232.11002, 231.98001,
        231.78    ],
        [231.38    , 231.38    , 231.38    , ..., 231.38    , 231.38    ,
        231.38    ]]],
  mask=False,
  fill_value=1e+20,
  dtype=float32)
```

Now do you remember the units on the air temperature? Look up above for air. it is kelvin! Now what real scientist uses Kelvin?
i am going to convert to good ole fahrenheit!

```
In [38]: air=air-273.15
air=air*9/5+32
```

But how does the air temperature work? Lets look at it and figure it out.

```
In [39]: np.shape(air)
```

```
Out[39]: (1, 73, 144)
```

```
In [40]: print(np.shape(lat))
print(np.shape(lon))
print(np.shape(time))
print(np.shape(air))
```

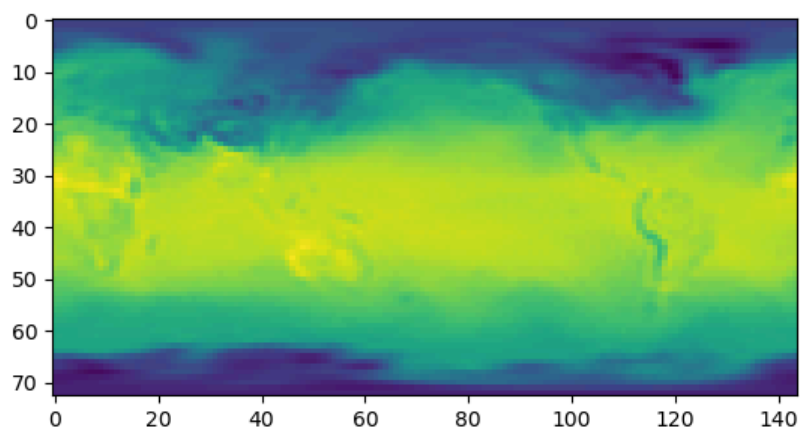
```
(73,)
(144,)
(1,)
(1, 73, 144)
```

It is a three dimensional array. It has only 1 data dimension and is then 73 by 144. Remember 73 is latitude. So it has 73 entries north south and then 144 east west. So it is a big array of temperatures. Sort of like when we did the "Brian.csv" lab!
Lets do the simplest thing to look at it.

```
In [41]: fig,ax=plt.subplots()
ax.imshow(air[0])
```

```
Out[41]: <matplotlib.image.AxesImage at 0x13fff85e0>
```


Figure

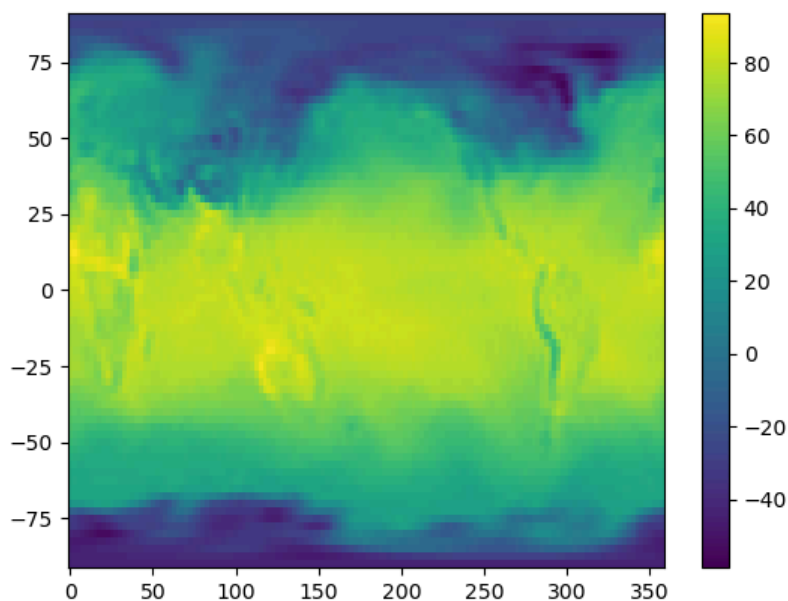


I can sort of pick out the continents but the axes make no sense. Instead we can use `pcolormesh` with the `lon` and `lat` to make a plot. Then we can add a colorbar.

```
In [42]: fig,ax=plt.subplots()
         pc=ax.pcolormesh(lon,lat,air[0])
         fig.colorbar(pc)
```

```
Out[42]: <matplotlib.colorbar.Colorbar at 0x14008c0d0>
```

Figure



Let's make a real map instead!

- Grab your map from above

- Just show the `ax.coastline()` and countries.
- Now we are going to add countouf. the f is for filled.
- Here is the example https://scitools.org.uk/cartopy/docs/v0.13/matplotlib/advanced_plotting.html
- add lon, lat, air[0], transform, choose your cmap, and levels.
- I did a lot of levels to really smooth it.

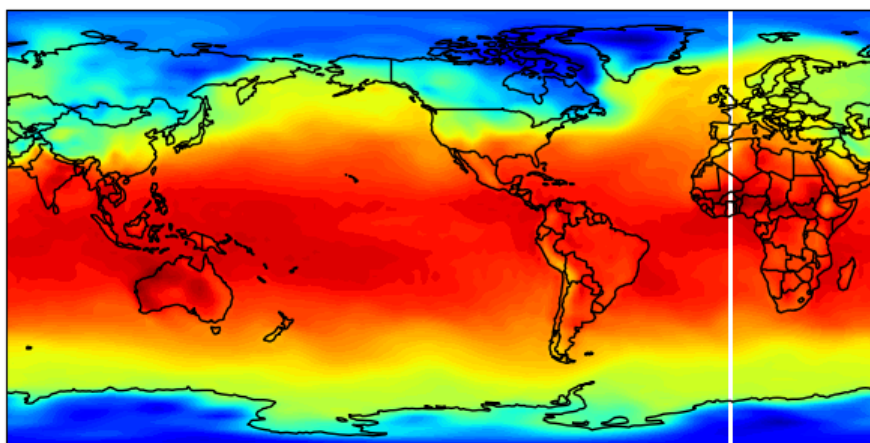
```
In [43]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

#ax = plt.axes(projection=ccrs.PlateCarree())
ax.coastlines()
ax.add_feature(cartopy.feature.BORDERS)

ax.contourf(lon, lat, air[0],
            transform=ccrs.PlateCarree(),
            cmap='jet', levels=100)
```

Out[43]: <cartopy.mpl.contour.GeoContourSet at 0x1401d2bb0>

Figure



That looks sharp! We will fix the white line in a bit. First Lets now add a colorbar. There are two ways to add a colorbar. This is way 1.

- Set the contourf equal to something.
- add a new set of axes where you want tthe color bar.
- turn on the colorbar
- add a title

```
In [77]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

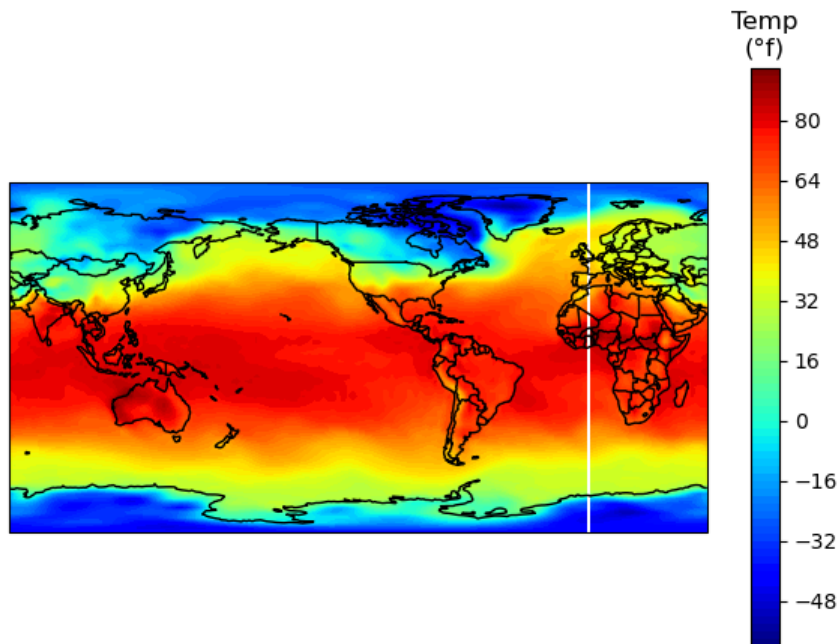
ax.coastlines()
ax.add_feature(cartopy.feature.BORDERS)

air_contour=ax.contourf(lon, lat, air[0],
                        transform=ccrs.PlateCarree(),
                        cmap='jet', levels=100)

cbar_ax=fig.colorbar(air_contour,ax=ax)
cbar_ax.ax.set_title('Temp\n(N{DEGREE SIGN}f)')
```

Out[77]: Text(0.5, 1.0, 'Temp\n(°f)')

Figure



Make the colorbar smaller

```
In [79]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

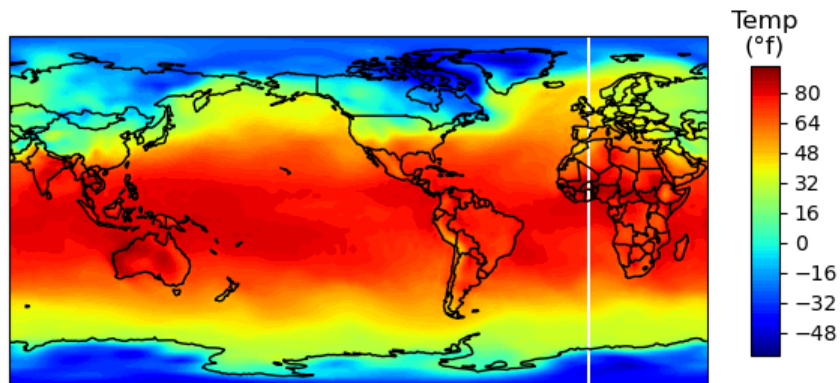
ax.coastlines()
ax.add_feature(cartopy.feature.BORDERS)

air_contour=ax.contourf(lon, lat, air[0],
                        transform=ccrs.PlateCarree(),
                        cmap='jet',levels=100)

cbar_ax=fig.colorbar(air_contour,ax=ax,shrink=0.5, aspect=20*0.5) #shrink is the size and aspect keeps the
cbar_ax.ax.set_title('Temp\n(\N{DEGREE SIGN}f)')
```

Out[79]: Text(0.5, 1.0, 'Temp\n(°f)')

Figure



If you want to get rid of the white line you can google it. <https://stackoverflow.com/questions/56348136/white-line-in-contour-plot-in-cartopy-on-center-longitude>

- I added the one line
- I used "data" in contourf
- You need to repeat the data where the world comes back together
- add cyclic accounts for the data coming full circle. Bascially saying the beginning and end should meet

```
In [94]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

ax.coastlines()
ax.add_feature(cartopy.feature.BORDERS)
ax.gridlines(draw_labels=True) #see if you like them.

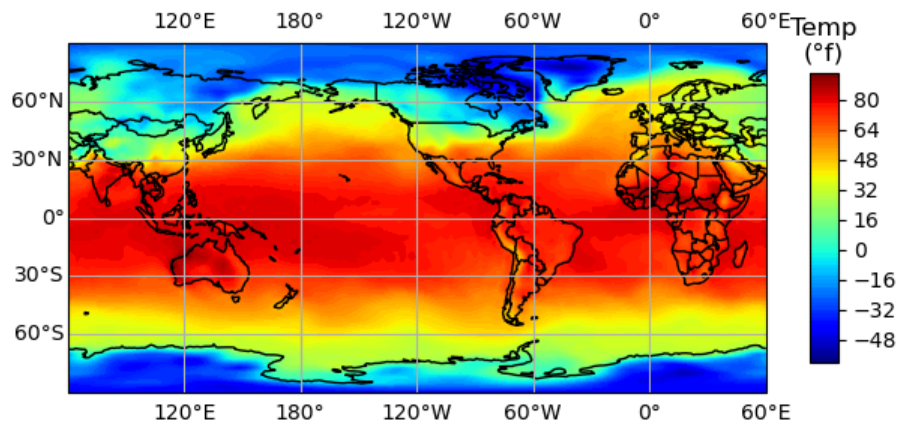
data, lonW = add_cyclic_point(air[0], coord=lon)

air_contour=ax.contourf(lonW, lat, data,
                        transform=ccrs.PlateCarree(),
                        cmap='jet',levels=100)

cbar_ax=fig.colorbar(air_contour,ax=ax,shrink=0.5, aspect=20*0.5) #shrink is the size and aspect keeps the
cbar_ax.ax.set_title('Temp\n(\N{DEGREE SIGN}f)')
```

```
Out[94]: Text(0.5, 1.0, 'Temp\n(°f)')
```

Figure



Remember that you can choose whatever color map you want.

<https://matplotlib.org/stable/users/explain/colors/colormaps.html>

```
In [81]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

ax.coastlines()
ax.add_feature(cartopy.feature.BORDERS)

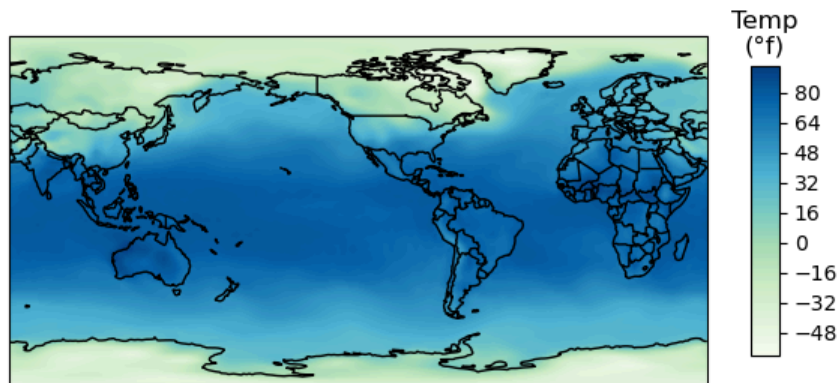
data, lonW = add_cyclic_point(air[0], coord=lon)

cmap='GnBu'
air_contour=ax.contourf(lonW, lat, data,
                        transform=ccrs.PlateCarree(),
                        cmap=cmap, levels=100)

cbar_ax=fig.colorbar(air_contour,ax=ax,shrink=0.5, aspect=20*0.5) #shrink is the size and aspect keeps the
cbar_ax.ax.set_title('Temp\n(\N{DEGREE SIGN}f)')

Out[81]: Text(0.5, 1.0, 'Temp\n(°f)')
```

Figure



That is a professional looking map. Nice job.

BUT

What if we want to change the color range. There are two ways and they do slightly different things.

Way 1. Change vmin and vmax. This will rescale the coloramp. play with the numbers.

```
In [84]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

ax.coastlines()
ax.add_feature(cartopy.feature.BORDERS)

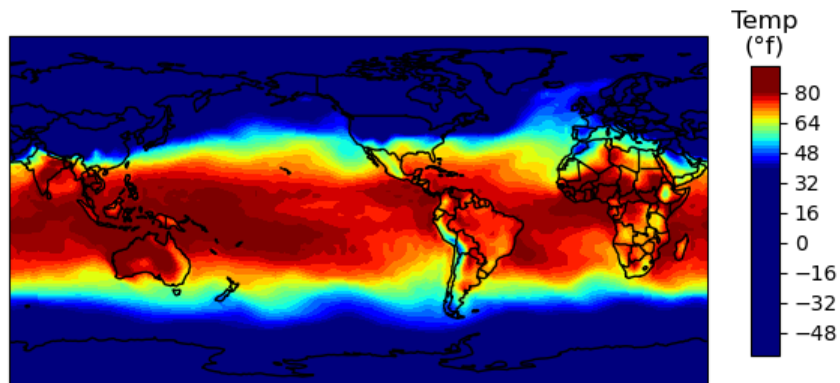
data, lonW = add_cyclic_point(air[0], coord=lon)

cmap='jet'
air_contour=ax.contourf(lonW, lat, data,
                        transform=ccrs.PlateCarree(),
                        cmap=cmap, levels=100, vmin=40, vmax=80)

cbar_ax=fig.colorbar(air_contour,ax=ax,shrink=0.5, aspect=20*0.5) #shrink is the size and aspect keeps the
cbar_ax.ax.set_title('Temp\n(N{DEGREE SIGN}f)')

Out[84]: Text(0.5, 1.0, 'Temp\n(°f)')
```

Figure



Way 2

Change the levels and where the color begins and ends using levels and linspace.

```
In [85]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.PlateCarree(central_longitude=-120)})
fig.set_size_inches(7.5,5)

ax.coastlines()
ax.add_feature(cartopy.feature.BORDERS)

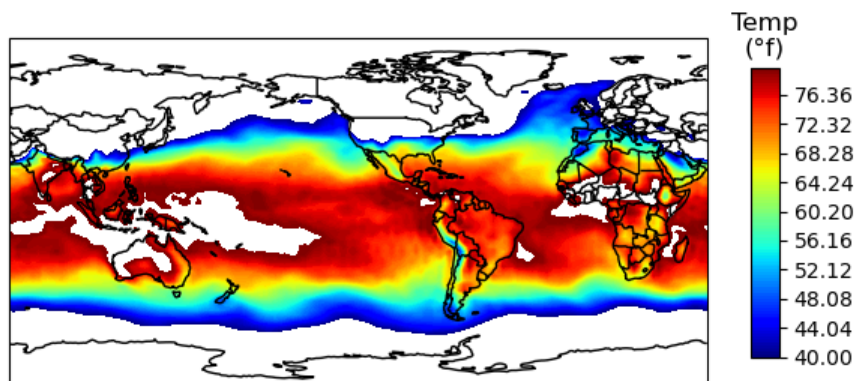
data, lonW = add_cyclic_point(air[0], coord=lon)

cmap='jet'
levels=np.linspace(40,80,100) # rememebr it is start, stop, number of values in the array so how smooth.
air_contour=ax.contourf(lonW, lat, data,
                        transform=ccrs.PlateCarree(),
                        cmap=cmap,levels=levels)

cbar_ax=fig.colorbar(air_contour,ax=ax,shrink=0.5, aspect=20*0.5) #shrink is the size and aspect keeps the
cbar_ax.ax.set_title('Temp\n(\N{DEGREE SIGN}f)')

Out[85]: Text(0.5, 1.0, 'Temp\n(°f)')
```

Figure



Both are good methods.

It just depends what you are trying to show.

Let's just look at the US on my birthday!

I am going to change the projection that I like more for the US.

My mom says it was cold.....

So we will need to adjust the levels

```
In [93]: fig,ax=plt.subplots(subplot_kw={"projection":ccrs.AlbersEqualArea(central_longitude=-100)})
fig.set_size_inches(7.5,5)

ax.coastlines()
ax.add_feature(cartopy.feature.BORDERS)
ax.add_feature(cartopy.feature.STATES)
ax.set_extent([-130,-60,20,46])
ax.gridlines(draw_labels=True) #see if you like them.

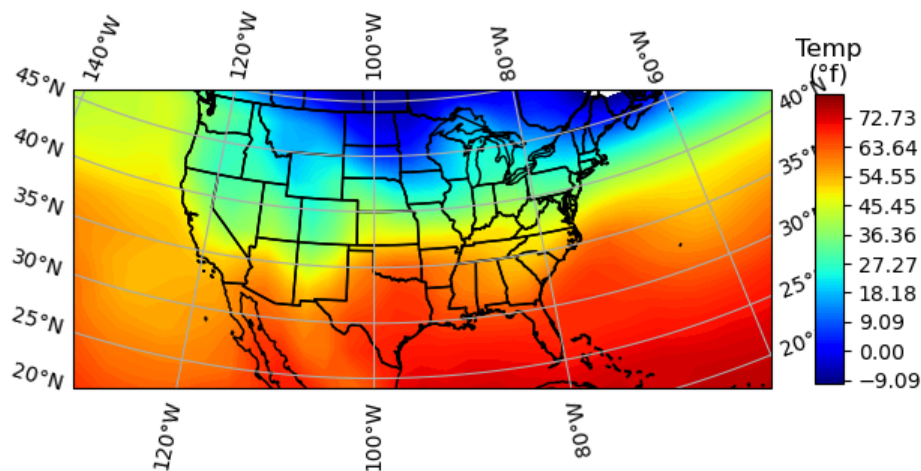
data, lonW = add_cyclic_point(air[0], coord=lon)

cmap='jet'
levels=np.linspace(-10,80,100) # rememebr it is start, stop, number of values in the array so how smooth.
air_contour=ax.contourf(lonW, lat, data,
                        transform=ccrs.PlateCarree(),
                        cmap=cmap,levels=levels)

cbar_ax=fig.colorbar(air_contour,ax=ax,shrink=0.5, aspect=20*0.5) #shrink is the size and aspect keeps the
cbar_ax.ax.set_title('Temp\n(\N{DEGREE SIGN}f)')
```

```
Out[93]: Text(0.5, 1.0, 'Temp\n(°f)')
```


Figure

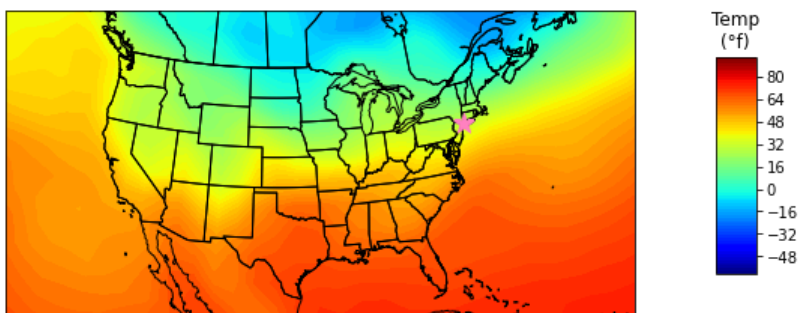


Now add a star (*) to where you were born

- Choose a fun xkcd color
- make the symbol big
- make it a star! or something else fun
- If you were born in another part of the world show it off!!!! If you don't want me to know; lie and make something up.
- If your star is getting stuck behind some drawings add the key worder "zorder=10". zorder tells it where to plot on top of each other and the higher number is on top.

In [109...

Out[109... Text(0.5, 1.0, 'Temp\n(°f)')



Time!

That is a great map. But to understand if we did thing correctly we need to understand the time. (add bad time pun here). So Let's add a nice date to the graph that we get from the netcdf file.

In [95]: `print (f.variables['time'])`

```
<class 'netCDF4._netCDF4.Variable'>
float64 time(time)
  units: hours since 1800-1-1 00:00:0.0
  long_name: Time
  actual_range: [1509000. 1509000.]
  delta_t: 0000-01-00 00:00:00
unlimited dimensions: time
current shape = (1,)
filling on, default _FillValue of 9.969209968386869e+36 used
```

What does this mean???? Time since 1800. In hours! Holy crazy. Lets look. But excel does this also. You choose a starting date and just keep track of the days since that time. I think excel starts at 1/1/1900. Lets check and see if it makes sense. The number of hours is 1509000. Divide that by 365/24 and lets see if it is year until when I was born.

```
In [31]: time
```

```
Out[31]: masked_array(data=[1509000.],
                      mask=False,
                      fill_value=1e+20)
```

```
In [32]: 1509000/365/24
```

```
Out[32]: 172.26027397260273
```

Luckily python offers ways to deal with it.

Luckily Python has a datetime module!!! We have been using it in Pandas w/o you realizing it.

<https://docs.python.org/2/library/datetime.html>

```
In [ ]: datetime.
```

We can use dateime and make a datetime parameter. Just like we have strings, ints, and floats we can make a datetime that we can play with

So lets add the year,month,day and see what happens

```
In [96]: datetime.datetime(2024,4,1)
```

```
Out[96]: datetime.datetime(2024, 4, 1, 0, 0)
```

```
In [97]: today=datetime.datetime(2025,4,7)
print (today)
type(today)
```

```
2025-04-07 00:00:00
```

```
Out[97]: datetime.datetime
```

See how the type is a datetime!

So we can make ourselves a datetime! plus datetime has a function called timedelta.... It figures out how long given leap years and all that! You create a timedelta type. So it tells you how long

```
In [98]: ?datetime.timedelta
```

```
Init signature: datetime.timedelta(self, /, *args, **kwargs)
```

```
Docstring:
```

```
Difference between two datetime values.
```

```
timedelta(days=0, seconds=0, microseconds=0, milliseconds=0, minutes=0, hours=0, weeks=0)
```

```
All arguments are optional and default to 0.
```

```
Arguments may be integers or floats, and may be positive or negative.
```

```
File: ~/anaconda3/envs/BigDataPython2025/lib/python3.8/datetime.py
```

```
Type: type
```

```
Subclasses: _Timedelta
```

so we need to make a timedelta. Remember delta means change in math terminology!

```
In [99]: datedelta=datetime.timedelta(hours=1509000)
print (datedelta)
```

```
print(type(datedelta))
```

```
62875 days, 0:00:00
<class 'datetime.timedelta'>
```

So timedelta is an amount of time. so we can put in our value and get a timedelta

```
In [100... datedelta=datetime.timedelta(hours=time[0])
print (datedelta)
print(type(datedelta))
```

```
62875 days, 0:00:00
<class 'datetime.timedelta'>
```

We can now do time math! This is crazy again. But we first need to set a start date

```
In [101... startdate=datetime.datetime(1800,1,1)
print (startdate)
print(type(startdate))
```

```
1800-01-01 00:00:00
<class 'datetime.datetime'>
```

Now we can just add them.

```
In [102... print (startdate+datedelta)
```

```
1972-02-24 00:00:00
```

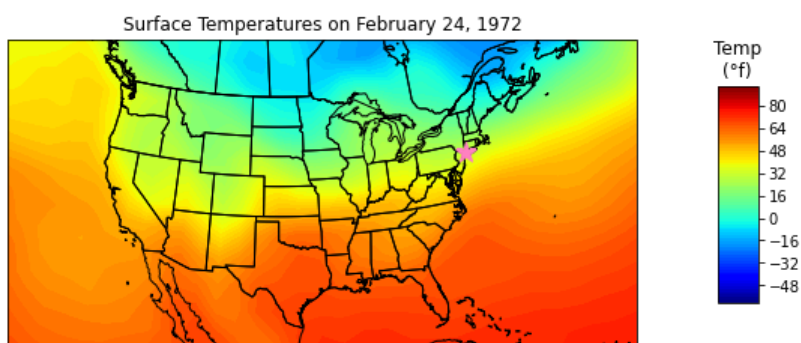
Lets set the date and print it nicely. Here is a summary of printing dates <https://stackoverflow.com/questions/311627/how-to-print-a-date-in-a-regular-format>

```
In [105... mapdate=startdate+datedelta
print ("The date is {:%A %B %d, %Y}".format(mapdate))
```

```
The date is Thursday February 24, 1972
```

So lets go back and make our map look great! Add a title. Save it and send to family or friends! Ask someone who remembers when you wer born if the temperature makes sense. Plus hand it in for homework. Your map must be from your day with your locattion with your title and scaled so we can easily see the temperature

```
In [115...
```



```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

In []:

In []:

Answers

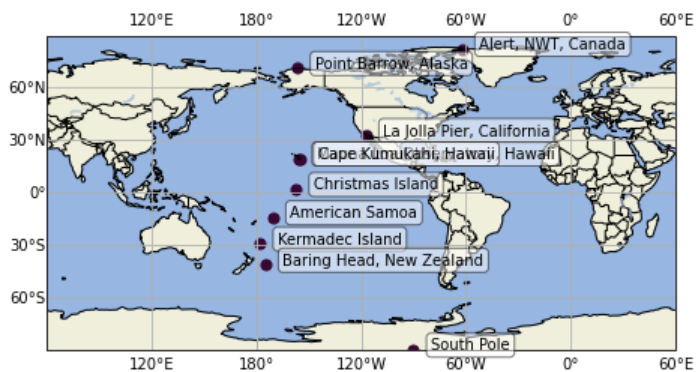
```
In [16]: props=dict(boxstyle='round',facecolor='white',alpha=0.5)

fig,ax=plt.subplots()
fig.set_size_inches(7.5,5)

ax = plt.axes(projection=ccrs.PlateCarree(central_longitude=-120))
ax.add_feature(cartopy.feature.LAND)
ax.add_feature(cartopy.feature.OCEAN)
ax.add_feature(cartopy.feature.COASTLINE)
ax.add_feature(cartopy.feature.BORDERS)
ax.add_feature(cartopy.feature.LAKES, alpha=0.5)
ax.gridlines(draw_labels=True)
ax.scatter(df.Longitude,df.Latitude,color='xkcd:eggplant',
           ,transform=ccrs.PlateCarree(),s=50)

for idx,dfr in df.iterrows():
    ax.text(dfr['Longitude']+10,dfr['Latitude'],dfr['StationName'],transform=ccrs.PlateCarree(),bbox=props)

ax.set_global()
```



In []: